SGCC

国网四川电力
State Grid Sichuan Electric Power

课时2: RPAR实例与Acting详解

1. RPAR的裁剪与实例化 (15分钟)

培训项目 AI种子团队培训教材
课程阶段 Day 1: Agentic AI 设计范式
生成日期 2026年03月22日

目录

- 课时2: RPAR实例与Acting详解
- 1. RPAR的裁剪与实例化 (15分钟)
- 1.1 核心概念
- 1.2 ReAct: R–A–R (简化版)
- 完整概念解释
- 结构特点
- 完整案例1：简单查询
- 完整案例2：多步推理
- 完整案例3：错误修正
- 伪代码实现
- 优缺点分析
- 1.3 Plan–and–Execute: R–P–A (规划版)
- 为什么需要规划阶段？
- 结构特点
- 规划策略对比
- 完整案例1：订票场景
- 完整案例2：数据处理场景
- 与ReAct的对比决策树
- 伪代码实现
- 1.4 完整RPAR: R–P–A–R (完全体)

课时2: RPAR实例与Acting详解

时长: 40分钟
主题: RPAR框架的实例化与Acting执行层详解
目标字数: 8,000字

---

1. RPAR的裁剪与实例化 (15分钟)

1.1 核心概念

引出裁剪概念:

> "刚才讲了RPAR的四个环节，但你们可能会想：每个任务都要走完整四个环节吗？那太麻烦了。实际上，不同场景可以用不同的'裁剪版'。就像你去超市买菜，不需要写详细的购物清单（Planning），直接看想买什么就拿什么。但如果你装修房子，就需要先规划好要买什么。"

核心思想:

> "RPAR是'完全体'，但不同场景可以用'裁剪版'。裁剪的原则是：**简单任务简化，复杂任务完整**。

1.2 ReAct: R–A–R (简化版)

完整概念解释

什么是ReAct?

ReAct（Reasoning + Acting）是由普林斯顿大学的研究人员在2022年提出的框架，核心论文是《ReAct: Synergizing Reasoning and Acting in Language Models》（Yao et al., 2022）。

核心思想: 将推理（Reasoning）和行动（Acting）结合起来，让大语言模型能够交替进行思考和执行，通过观察环境反馈来不断调整策略。

核心作者: – Shunyu Yao（普林斯顿大学） – Jeffrey Zhao（Google Research） – Dian Yu（Google Research） – Nan Du（Google Research） – Izhak Shafran（Google Research） – Karthik Narasimhan（普林斯顿大学） – Yuan Cao（Google Research） – Kamel R. Arabshahi（Google Research）

论文核心贡献: 1. 证明了将推理和行动结合优于单独的推理或行动 2. 在知识密集型推理任务上取得了显著提升 3. 提供了可解释的执行轨迹，便于调试和验证

结构特点

ReAct结构: Reasoning → Acting → Observation → Reasoning (循环)

```
ReAct循环:

┌──────────────────────────────────────────────────────┐
│  Thought (思考)                                      │
│  - 分析当前状态                                      │
│  - 决定下一步行动                                    │
└──────────────────────────────────────────────────────┘
                    ↓
┌──────────────────────────────────────────────────────┐
│  Action (行动)                                       │
│  - 调用工具                                          │
│  - 执行操作                                          │
└──────────────────────────────────────────────────────┘
                    ↓
┌──────────────────────────────────────────────────────┐
│  Observation (观察)                                  │
│  - 获取执行结果                                      │
│  - 环境反馈                                          │
└──────────────────────────────────────────────────────┘
                    └──────────────→ 回到Thought
```

特点: – ✅ 去掉了显式Planning（规划隐含在Reasoning中） – ✅ 去掉了显式Reflection（反思隐含在Reasoning中） – ✅ 边想边做，简单直接 – ✅ 适合快速响应的场景

适用场景: – 简单任务 – 不需要复杂规划 – 边想边做即可 – 对响应速度要求高

完整案例1：简单查询

场景: 查询今天的天气

```
用户："查一下今天的天气"

ReAct执行过程:

Step 1 - Thought:
用户需要天气信息，我需要查询今天的天气数据。
我需要调用天气查询API。

Step 2 - Action:
调用 get_weather API
参数: {date: "today", location: "current"}

Step 3 - Observation:
{
  "temperature": "15-22°C",
  "condition": "多云",
  "humidity": "65%",
```

```
  "wind": "东南风2级"
}

Step 4 - Thought:
已获得天气数据，可以回答用户了。
数据完整，不需要进一步查询。

Step 5 - Action:
生成回答

Final Answer:
"今天多云，气温15-22°C，湿度65%，东南风2级。适合户外活动，建议带件薄外套。"
```

分析: – 简单直接的R–A–R循环 – 一轮交互即完成 – 不需要复杂规划 – 响应快速

完整案例2：多步推理

场景: 查询某项目投资额并计算与预算的差异

```
用户："查一下PRJ-2024-001项目的投资情况，看看超支了吗？"

ReAct执行过程：

Step 1 - Thought:
用户想了解PRJ-2024-001项目的投资情况，并与预算对比。
我需要：
1. 查询项目实际投资额
2. 查询项目预算
3. 计算差异
4. 判断是否有超支

让我先查询项目实际投资额。

Step 2 - Action:
调用 query_project_investment API
参数：{project_id: "PRJ-2024-001"}

Step 3 - Observation:
{
  "project_id": "PRJ-2024-001",
  "project_name": "高新区配网改造",
  "actual_investment": 2850,
  "currency": "万元",
  "status": "进行中"
}

Step 4 - Thought:
已获取实际投资额：2850万元。
现在需要查询预算。

Step 5 - Action:
调用 query_project_budget API
参数：{project_id: "PRJ-2024-001"}
```

```
Step 6 - Observation:
{
  "project_id": "PRJ-2024-001",
  "budget": 2500,
  "currency": "万元"
}

Step 7 - Thought:
已获得预算：2500万元。
实际投资：2850万元。
差异：2850 - 2500 = 350万元（超支14%）

需要计算并生成分析结果。

Step 8 - Action:
计算差异百分比

Step 9 - Observation:
计算结果：超支14%

Step 10 - Thought:
已计算出超支比例。需要生成完整的回答。

Final Answer:
"PRJ-2024-001项目（高新区配网改造）投资情况分析：

📊 数据对比
• 预算：2,500万元
• 实际投资：2,850万元
• 差异：+350万元（超支14%）

⚠️ 风险提醒
项目已超支14%，建议：
1. 审查超支原因
2. 评估后续资金需求
3. 如需追加预算，尽快启动审批流程

项目状态：进行中"
```

分析: - 多轮R-A-R循环 - 每步都有明确的Thought - 根据Observation动态调整下一步 - 逐步完成复杂查询

完整案例3：错误修正

场景: 查询某区域数据，第一次查询参数错误

```
用户："查一下天府新区的用电数据"

ReAct执行过程：

Step 1 - Thought:
用户需要天府新区的用电数据。
```

```
我需要调用用电数据查询API。

Step 2 - Action:
调用 query_electricity_data API
参数: {region: "天府新区"}

Step 3 - Observation:
{
  "error": true,
  "error_code": "REGION_NOT_FOUND",
  "message": "区域'天府新区'不存在，是否指'天府新区直管区'? "
}

Step 4 - Thought:
查询失败，错误信息提示区域名称可能不对。
系统建议使用'天府新区直管区'。
让我重试用正确的区域名称。

Step 5 - Action:
调用 query_electricity_data API
参数: {region: "天府新区直管区"}

Step 6 - Observation:
{
  "region": "天府新区直管区",
  "date": "2024-03-01",
  "total_usage": 5678.5,
  "unit": "MWh",
  "peak_load": 1250,
  "user_count": 45678
}

Step 7 - Thought:
查询成功！已获得天府新区直管区的用电数据。
数据完整，可以回答用户。

Final Answer:
"天府新区直管区用电数据 (2024-03-01):
• 总用电量: 5,678.5 MWh
• 峰值负荷: 1,250 MW
• 用户数量: 45,678户

注: 系统区域名称为'天府新区直管区'。"
```

分析: – 展示了ReAct的自我纠错能力 – 遇到错误不终止，而是分析原因 – 根据Observation调整策略 – 体现了Reasoning环节的反思特性

伪代码实现

```
class ReActAgent:
    """
    ReAct(Reasoning + Acting)智能体实现类。

    该类实现了经典的ReAct模式，通过交替进行推理(Reasoning)和行动(Acting)
```

```
来完成复杂任务。每轮迭代中，Agent首先思考下一步行动，然后执行工具，
最后观察结果并更新上下文。这种范式特别适合需要多步骤推理的电力业务场景，
如故障诊断、数据分析、报告生成等。

Attributes:
    tools (dict): 可用工具集合，键为工具名，值为工具函数
        示例: {"query_data": query_func, "analyze": analyze_func}
    llm (LLM): 大语言模型实例，用于生成Thought和Answer
    max_rounds (int): 最大执行轮次限制，防止无限循环

Examples:
    >>> # 初始化Agent
    >>> tools = {
    ...     "query_weather": get_weather,
    ...     "query_electricity": get_electricity_data,
    ...     "calculate": calculator
    ... }
    >>> agent = ReActAgent(tools, llm_client)

    >>> # 执行查询任务
    >>> result = agent.run("查询成都今天用电负荷")
    >>> print(result)
    '成都今日用电负荷为1250MW，较昨日上升5%...'

    >>> # 复杂任务：故障诊断
    >>> result = agent.run("分析武侯区供电异常原因", max_rounds=15)

Note:
    该类是教学示例实现，实际生产环境应考虑：
    1. 添加超时控制机制
    2. 实现更健壮的错误恢复
    3. 支持并行工具调用
    4. 添加执行日志和监控
"""

def __init__(self, tools: dict, llm):
    """
    初始化ReAct Agent。

    Args:
        tools (dict): 工具函数字典，格式为{tool_name: callable}
            每个工具函数应接受关键字参数并返回执行结果
        llm: 大语言模型客户端实例，需实现generate方法

    Raises:
        ValueError: 当tools为空字典或包含非callable值时抛出
        TypeError: 当llm未实现generate方法时抛出
    """
    if not tools:
        raise ValueError("工具集合不能为空，至少需要一个可用工具")
    if not all(callable(tool) for tool in tools.values()):
        raise ValueError("所有工具必须是可调用的函数或方法")

    self.tools = tools
    self.llm = llm

def run(self, user_input: str, max_rounds: int = 10) -> str:
```

```
"""
执行ReAct主循环，处理用户请求。

该方法实现了核心的ReAct循环逻辑：
1. Thought：基于当前上下文生成思考
2. Action：解析并执行工具调用
3. Observation：获取执行结果并更新上下文
循环直到任务完成或达到最大轮次。

Args:
    user_input (str)：用户的自然语言请求
        示例："查询成都高新区今天的用电负荷"
    max_rounds (int, optional)：最大执行轮次，防止无限循环
        默认值：10
        建议值：简单任务5-8轮，复杂任务10-15轮

Returns:
    str：Agent生成的最终回答，包含完整的推理过程总结

Raises:
    RuntimeError：当LLM调用失败或工具执行出现严重错误时
    TimeoutError：当执行时间超过预设限制时（如实现了超时控制）

Examples:
    >>> # 简单查询
    >>> agent.run("今天天气如何？")
    '根据查询结果，今天成都天气晴朗，气温18-25℃...'

    >>> # 电力业务：负荷分析
    >>> query = "分析最近7天武侯区用电趋势"
    >>> response = agent.run(query, max_rounds=12)
    >>> print(response)

    >>> # 故障诊断场景
    >>> query = "诊断CD-001线路异常原因"
    >>> result = agent.run(query, max_rounds=15)

Note:
    - 每轮都会消耗LLM Token，复杂任务成本较高
    - 当前实现为串行执行，不支持并行工具调用
    - 建议在实际应用中添加重试机制和降级策略
"""
context = f"用户请求：{user_input}\n\n"

for round_num in range(max_rounds):
    # Step 1: Reasoning - 思考下一步
    thought = self.llm.generate(
        prompt=f"{context}\n基于以上信息，思考下一步应该做什么？",
        system_prompt="你是一个智能助手，需要分析问题并决定下一步行动。"
    )
    context += f"思考：{thought}\n"

    # Step 2: 判断是否完成任务
    if "无需行动" in thought or "任务完成" in thought:
        answer = self.llm.generate(
            prompt=f"{context}\n请基于以上思考过程，生成最终回答。"
```

```
            )
            return answer

        # Step 3: Acting - 解析并执行行动
        action_parsed = self.parse_action(thought)
        if action_parsed:
            tool_name = action_parsed["tool"]
            parameters = action_parsed["parameters"]

            # Step 4: 执行工具并获取观察结果
            if tool_name in self.tools:
                try:
                    result = self.tools[tool_name](**parameters)
                    observation = f"执行结果: {result}"
                except Exception as e:
                    observation = f"执行错误: {str(e)}"
            else:
                observation = f"错误: 未找到工具 '{tool_name}'"

            context += f"行动: 调用 {tool_name}\n"
            context += f"观察: {observation}\n"
        else:
            context += "未解析到有效行动\n"

    # 达到最大轮次，强制结束
    return self.llm.generate(
        prompt=f"{context}\n已达到最大执行轮次，请基于当前信息给出最佳回答。"
    )

def parse_action(self, thought: str) -> dict | None:
    """
    从LLM生成的Thought中解析出可执行的行动。

    该方法负责将自然语言的思考文本转换为结构化的行动指令。
    实际实现可以采用:
    1. 结构化输出: 要求LLM按JSON格式返回Action
    2. 正则表达式: 从文本中提取工具名和参数
    3. Few-shot示例: 提供解析示例指导LLM

    Args:
        thought (str): LLM生成的思考文本
            示例: "我需要查询成都今天的天气，调用query_weather工具"

    Returns:
        dict | None: 解析成功返回行动字典，失败返回None
            格式: {"tool": str, "parameters": dict}
            示例: {"tool": "query_weather", "parameters": {"city": "成都"}}

    Raises:
        ValueError: 当thought为空字符串时抛出

    Examples:
        >>> # 标准Action格式
        >>> thought = "调用query_data工具，参数region='成都'"
        >>> action = agent.parse_action(thought)
        >>> action["tool"]
        'query_data'
```

```
        >>> # 无Action情况
        >>> thought = "我已经有足够的信息回答问题"
        >>> agent.parse_action(thought) is None
        True

    TODO:
        实际生产环境应实现具体的解析逻辑，例如：
        - 使用正则表达式匹配 "调用(.*?)工具"
        - 使用JSON解析器提取结构化数据
        - 调用专门的解析模型处理
    """
    if not thought:
        raise ValueError("思考文本不能为空")

    # 示例实现占位
    # 实际应用中的解析逻辑示例：
    # 1. 检查是否包含工具调用意图
    # 2. 使用正则提取工具名和参数
    # 3. 验证工具存在性
    # 4. 返回结构化数据

    # 假设Thought格式："调用{tool_name}，参数{json_params}"
    # 解析出：{"tool": tool_name, "parameters": json.loads(params)}
    pass

# 使用示例
tools = {
    "query_weather": get_weather,
    "query_electricity": get_electricity_data,
    "calculate": calculator
}

agent = ReActAgent(tools, llm)
result = agent.run("查一下成都今天的天气")
```

优缺点分析

优点:

优点	说明
简单直观	概念简单，易于理解和实现
响应快速	不需要复杂规划，直接执行
灵活应变	根据观察动态调整，适应性好
可解释性强	每一步都有Thought，便于调试
适合简单任务	对于简单查询效率很高

缺点:

缺点	说明
无显式规划	复杂任务的规划能力弱
效率不稳定	可能陷入低效循环
难以并行	无法利用并行执行优化效率
长任务困难	任务复杂时轮次过多
资源消耗	每步都需要LLM推理，Token消耗大

适用场景总结: – ✅ 简单查询类任务 – ✅ 单步或少量步骤的任务 – ✅ 需要灵活应变的场景 – ✅ 对响应速度要求高的场景 – ❌ 复杂的多步骤任务 – ❌ 需要严格规划的场景 – ❌ 需要并行执行的场景

1.3 Plan-and-Execute: R-P-A (规划版)

为什么需要规划阶段?

ReAct的问题:

```
ReAct的问题场景:
用户: "帮我订一张北京到上海的机票，要上午的"

ReAct执行:
Step 1: 查询北京到上海的航班
Step 2: 发现有很多航班，随便选一个?
Step 3: 等等，用户要上午的，需要筛选
Step 4: 筛选后发现上午航班很少，可能不符合要求
Step 5: 需要重新查询其他日期?
...

问题:
- 边做边想，可能走弯路
- 没有全局视角
- 效率低下
- 可能陷入死循环
```

Plan-and-Execute的解决方案:

```
Plan-and-Execute执行:

Planning阶段（一次性）:
1. 查询北京到上海的航班（3月15日）
2. 筛选上午的航班（6:00-12:00）
3. 按价格排序
4. 选择最优选项
5. 预订机票
```

```
Executing阶段（批量执行）:
- 执行步骤1-5
- 如果发现没有上午航班，返回Planning阶段重新规划
```

核心价值: – 避免低效循环 – 一次性规划全局 – 明确执行路径 – 支持并行优化

结构特点

Plan-and-Execute结构: Reasoning → Planning → Acting(List)

```
┌──────────────────────────────────────────────────────────┐
│ 理解任务 → 分解步骤 → 排定优先级 → 输出计划 │
└──────────────────────────────────────────────────────────┘
```

特点: – ✅ 显式Planning阶段 – ✅ Acting批量执行（List表示一次性执行多个） – ✅ 去掉了显式Reflection（通过重规划实现调整） – ✅ 适合步骤明确的任务

适用场景: – 步骤明确的任务 – 需要提前规划 – 批量执行更高效 – 可以并行处理

规划策略对比

策略1: 一次性规划

```
特点:
- 任务开始前完成全部规划
- 执行阶段严格按照计划
- 优点: 全局最优、可预测
- 缺点: 缺乏灵活性、无法应对变化

适用:
- 任务明确、环境稳定
- 步骤之间依赖关系清晰
- 不需要中途调整
```

策略2: 渐进式规划

```
特点:
- 先规划主要里程碑
- 执行一部分后再细化后续步骤
- 优点: 兼顾规划和灵活
- 缺点: 需要多次规划

适用:
- 任务复杂、需要分阶段
```

- 前期信息不充分
- 需要根据实际情况调整

对比表:

维度	一次性规划	渐进式规划
规划时机	任务开始前	分阶段进行
灵活性	低	高
全局性	高	中
执行效率	高（无中断）	中（需要重规划）
适用场景	明确任务	探索性任务

完整案例1：订票场景

场景: 帮我订一张北京到上海的机票，要上午的

```
用户："帮我订一张北京到上海的机票，要上午的"

Plan-and-Execute执行过程：

【Phase 1: Planning】

Reasoning:
用户需要预订北京到上海的机票，要求上午时段。
需要完成的任务：
1. 查询航班信息
2. 筛选上午航班（6:00-12:00）
3. 按价格和舒适度排序
4. 选择最优选项
5. 完成预订

Planning输出：
执行计划：
Step 1: 调用query_flights，查询北京到上海的航班
        参数：{from: "北京", to: "上海", date: "今天"}

Step 2: 调用filter_flights，筛选上午航班
        参数：{time_range: "06:00-12:00"}

Step 3: 调用sort_flights，按价格排序
        参数：{sort_by: "price", order: "asc"}

Step 4: 调用select_flight，选择最优航班
        参数：{criteria: "性价比最高"}

Step 5: 调用book_ticket，完成预订
        参数：{flight_id: "<选择的航班ID>"}
```

【Phase 2: Execution】

执行Step 1: query_flights
结果: 获取到12个航班信息

执行Step 2: filter_flights
结果: 筛选出4个上午航班

执行Step 3: sort_flights
结果: 排序完成

执行Step 4: select_flight
结果: 选择CA1501航班 (08:00-10:20, ¥680)

执行Step 5: book_ticket
结果: 预订成功

【结果输出】
"机票预订成功!

✈️ 航班信息
- 航班号: CA1501
- 时间: 08:00-10:20 (2小时20分钟)
- 价格: ¥680 (经济舱)
- 航司: 中国国际航空

🎫 预订确认
- 票号: 781-1234567890
- 乘客: 根据您的账户信息
- 状态: 已出票

请提前2小时到达机场办理登机手续。"

完整案例2: 数据处理场景

场景: 分析成都各区的用电数据, 生成对比报告

用户: "分析一下成都各区的用电数据, 生成对比报告"

Plan-and-Execute执行过程:

【Phase 1: Planning】

Reasoning:
用户需要分析成都各区的用电数据并生成对比报告。
这是一个数据分析任务, 需要:
1. 获取成都各区的用电数据
2. 计算各区的关键指标 (总量、增长率等)
3. 进行区域对比分析
4. 生成可视化图表
5. 撰写分析报告

```
Planning输出：
执行计划：

并行批次1（数据获取）：
├── Step 1.1: 查询高新区用电数据
├── Step 1.2: 查询天府新区用电数据
├── Step 1.3: 查询锦江区用电数据
├── Step 1.4: 查询青羊区用电数据
└── Step 1.5: 查询武侯区用电数据

顺序批次2（数据分析，依赖批次1）：
├── Step 2.1: 计算各区总量和增长率
├── Step 2.2: 区域排名分析
└── Step 2.3: 异常情况识别

并行批次3（可视化，依赖批次2）：
├── Step 3.1: 生成柱状图（各区用电量对比）
├── Step 3.2: 生成折线图（增长趋势）
└── Step 3.3: 生成热力图（区域分布）

顺序批次4（报告生成，依赖批次3）：
└── Step 4.1: 生成完整分析报告

【Phase 2: Execution】

批次1执行（并行）：
✓ 高新区数据: 12,345 MWh, 同比+5.2%
✓ 天府新区数据: 9,876 MWh, 同比+8.7%
✓ 锦江区数据: 8,765 MWh, 同比+2.1%
✓ 青羊区数据: 7,654 MWh, 同比+1.8%
✓ 武侯区数据: 10,234 MWh, 同比+4.5%

批次2执行：
✓ 指标计算完成
✓ 排名：高新区 > 武侯区 > 天府新区 > 锦江区 > 青羊区
✓ 异常：无

批次3执行（并行）：
✓ 柱状图生成
✓ 折线图生成
✓ 热力图生成

批次4执行：
✓ 分析报告生成

【结果输出】
"成都各区用电数据分析报告

📊 总体概况
• 分析区域：5个区
• 分析周期：2024年2月
• 总用电量：48,874 MWh
```

🏆 区域排名
1. 高新区：12,345 MWh（+5.2%）
2. 武侯区：10,234 MWh（+4.5%）
3. 天府新区：9,876 MWh（+8.7%）⭐ 增长最快
4. 锦江区：8,765 MWh（+2.1%）
5. 青羊区：7,654 MWh（+1.8%）

📈 关键发现
- 天府新区增长最快（+8.7%），反映区域发展迅速
- 中心城区（锦江、青羊）增长趋缓，趋于饱和
- 高新区作为经济中心，用电量稳居第一

📄 附件
- 区域对比图.png
- 增长趋势图.png
- 详细数据表.xlsx

完整报告已保存。"

与ReAct的对比决策树

```
选择ReAct还是Plan-and-Execute?

任务是否简单明确?
 │
 ├── 是 → 是否需要快速响应?
 │         │
 │         ├── 是 → 【ReAct】✓
 │         │              简单查询，边想边做
 │         │
 │         └── 否 → 进一步分析...
 │
 └── 否 → 步骤是否明确?
          │
          ├── 是 → 是否可以并行?
          │         │
          │         ├── 是 → 【Plan-and-Execute】✓
          │         │              先规划，批量执行
          │         │
          │         └── 否 → 【Plan-and-Execute】✓
          │                        先规划，顺序执行
          │
          └── 否 → 【完整RPAR】✓
                         需要试错和反思
```

详细对比表:

维度	ReAct	Plan-and-Execute
规划方式	隐式，边做边想	显式，先规划后执行
执行模式	单步执行	批量执行
并行支持	❌ 不支持	✅ 支持
灵活性	高	中
效率	中	高
适用任务	简单任务	步骤明确的复杂任务
典型场景	查询天气	数据分析、批量处理

伪代码实现

```
class PlanAndExecuteAgent:
    def __init__(self, tools, llm):
        self.tools = tools
        self.llm = llm

    def run(self, user_input, max_replans=3):
        """
        Plan-and-Execute主流程
        """
        context = f"用户请求：{user_input}\n\n"
        replan_count = 0

        while replan_count < max_replans:
            # Phase 1: Planning
            plan = self.planning_phase(context)
            context += f"执行计划：\n{self.format_plan(plan)}\n"

            # Phase 2: Execution
            results, need_replan = self.execution_phase(plan, context)

            if not need_replan:
                # 执行成功，整合结果
                final_answer = self.integrate_results(results, context)
                return final_answer
            else:
                # 需要重新规划
                context += f"执行遇到问题，需要重新规划。\n"
                replan_count += 1

        # 达到最大重规划次数
        return self.generate_partial_answer(context)

    def planning_phase(self, context):
        """
        规划阶段：生成执行计划
        """
        plan_prompt = f"""
{context}

请基于以上信息，制定详细的执行计划。
计划应该包含：
```

```
1. 需要执行的步骤列表
2. 每个步骤使用的工具
3. 步骤之间的依赖关系
4. 哪些步骤可以并行执行

请以JSON格式输出计划:
{{
    "steps": [
        {{
            "id": 1,
            "description": "步骤描述",
            "tool": "工具名称",
            "parameters": {{参数}},
            "dependencies": [依赖的步骤ID],
            "parallel_group": "并行组编号（可并行执行的步骤组）"
        }}
    ]
}}
"""
        plan_json = self.llm.generate(plan_prompt)
        return json.loads(plan_json)

    def execution_phase(self, plan, context):
        """
        执行阶段：按计划执行
        """
        results = {}
        need_replan = False

        # 按依赖关系组织步骤
        execution_groups = self.organize_by_dependencies(plan["steps"])

        for group in execution_groups:
            # 组内步骤并行执行
            group_results = self.execute_parallel(group)

            for step_id, result in group_results.items():
                results[step_id] = result

                # 检查是否需要重新规划
                if self.need_replan(result):
                    need_replan = True
                    context += f"步骤 {step_id} 执行异常：{result}\n"
                    break

            if need_replan:
                break

        return results, need_replan

    def execute_parallel(self, steps):
        """
        并行执行一组步骤
        """
        from concurrent.futures import ThreadPoolExecutor

        def execute_step(step):
            tool_name = step["tool"]
            parameters = step["parameters"]
```

```
            if tool_name in self.tools:
                try:
                    result = self.tools[tool_name](**parameters)
                    return step["id"], {"success": True, "data": result}
                except Exception as e:
                    return step["id"], {"success": False, "error": str(e)}
            else:
                return step["id"], {"success": False, "error": f"工具 {tool_name

        with ThreadPoolExecutor() as executor:
            futures = [executor.submit(execute_step, step) for step in steps]
            results = {}
            for future in futures:
                step_id, result = future.result()
                results[step_id] = result

        return results

    def need_replan(self, result):
        """
        判断是否需要重新规划
        """
        if not result.get("success", False):
            # 执行失败，可能需要重新规划
            error = result.get("error", "")
            # 某些错误可以重试，某些需要重新规划
            if "REGION_NOT_FOUND" in error or "INVALID_PARAMETER" in error:
                return True
        return False

    def integrate_results(self, results, context):
        """
        整合执行结果，生成最终回答
        """
        integrate_prompt = f"""
{context}

执行结果:
{json.dumps(results, ensure_ascii=False, indent=2)}

请基于以上信息，生成完整的回答。
"""
        return self.llm.generate(integrate_prompt)

# 使用示例
agent = PlanAndExecuteAgent(tools, llm)
result = agent.run("分析成都各区的用电数据，生成对比报告")
```

1.4 完整RPAR: R–P–A–R (完全体)

什么时候需要完整循环?

适用场景: – 复杂任务 – 需要试错调整 – 不确定性高 – 需要持续优化

试错机制详解

如何检测错误:

```
错误检测维度:
1. 结果完整性检查
   - 是否获取了所有必要数据?
   - 输出是否完整?

2. 结果合理性检查
   - 数值是否在合理范围?
   - 与预期是否相符?

3. 工具执行状态检查
   - API返回状态码
   - 异常捕获
```

如何回退:

```
回退策略:
1. 单步回退
   - 当前步骤失败，重试或换方案

2. 多步回退
   - 回到之前的某个环节
   - 基于依赖关系决定回退点

3. 完全重启
   - 回到Reasoning重新分析
   - 彻底调整策略
```

如何重新规划:

```
重新规划触发条件:
- 关键步骤失败
- 结果不符合预期
- 发现新的约束条件
- 环境发生变化

重新规划策略:
- 保留已成功的部分
- 调整失败的部分
- 优化整体路径
```

投资问效案例详细走3轮循环

场景: 分析某投资项目的效益情况

用户："分析一下PRJ-2024-015项目的投资效益"

完整RPAR执行过程：

```
═══════════════════════════════════════════════════════════════
Round 1
═══════════════════════════════════════════════════════════════

【Reasoning】
用户需要分析PRJ-2024-015项目的投资效益。
需要获取的信息：
- 项目基本信息
- 投资额
- 收益数据
- 进度信息

【Planning】
计划：
Step 1: 查询项目基本信息
Step 2: 查询投资数据
Step 3: 查询收益数据
Step 4: 计算ROI
Step 5: 生成分析报告

【Acting】
执行Step 1: 查询项目基本信息
结果: {
  "project_id": "PRJ-2024-015",
  "name": "智能配电网改造项目",
  "status": "进行中",
  "start_date": "2024-01-15"
}

执行Step 2: 查询投资数据
结果: {
  "total_investment": 3200,
  "currency": "万元",
  "paid_amount": 2400
}

执行Step 3: 查询收益数据
结果: {
  "error": "数据不存在",
  "message": "该项目尚未产生收益数据"
}

【Reflection】
⚠️ 发现关键问题：
- 项目状态为"进行中"
- 缺少收益数据（项目尚未完成）
- 无法计算完整ROI

评估结果：不完整，需要调整策略。
```

调整决策:
- 不能按原计划计算ROI
- 需要重新规划分析维度
- 改为分析: 进度、资金使用效率、预期效益

Round 2

【Reasoning】
重新分析当前情况:
- 项目进行中, 无法获取实际收益
- 需要调整分析维度
- 可用数据: 进度、投资、计划数据

新的分析目标:
- 项目进度分析
- 资金使用效率分析
- 基于计划的预期ROI分析

【Planning】
新计划:
Step 1: 查询项目详细进度数据
Step 2: 查询资金使用明细
Step 3: 查询项目计划和预期收益
Step 4: 计算进度完成率
Step 5: 计算资金使用效率
Step 6: 预测项目完成后的ROI
Step 7: 生成综合分析报告

【Acting】
执行Step 1: 查询进度数据
结果: {
```
  "planned_progress": 75,
  "actual_progress": 68,
  "delay_days": 15
}
```

执行Step 2: 查询资金使用明细
结果: {
```
  "budget": 3200,
  "used": 2400,
  "utilization_rate": 75
}
```

执行Step 3: 查询计划和预期收益
结果: {
```
  "planned_returns": 4100,
  "expected_completion": "2024-06-30"
}
```

执行Step 4-6: 计算分析
结果: {
```
  "progress_rate": "90.7%",
  "fund_efficiency": "正常",
```

```
  "expected_roi": "28.1%"
}
```

【Reflection】
✓ 数据获取完整
✓ 分析计算成功
✓ 但发现新情况：项目延期15天

需要进一步分析延期原因和影响。

评估结果：数据完整，但需要补充分析延期情况。

Round 3

【Reasoning】
已获得主要分析数据，但发现项目延期。
需要补充分析：
- 延期原因
- 延期对ROI的影响
- 风险评估

【Planning】
补充计划：
Step 1: 查询延期原因
Step 2: 评估延期影响
Step 3: 生成最终报告（包含所有分析维度）

【Acting】
执行Step 1: 查询延期原因
结果: {
```
  "reason": "设备交付延迟",
  "impact": "预计影响完工日期15天",
  "mitigation": "已协调供应商加急"
}
```

执行Step 2: 评估延期影响
结果: {
```
  "cost_impact": "增加成本约50万元",
  "roi_adjustment": "预计ROI降至26.5%"
}
```

执行Step 3: 生成报告
结果: 报告生成成功

【Reflection】
✓ 所有分析完成
✓ 数据完整
✓ 报告包含：
  - 项目概况
  - 进度分析（延期15天）
  - 资金使用分析
  - 预期ROI分析（26.5%，原预计28.1%）

  - 风险评估
  - 建议措施

评估结果：任务完成，可以输出最终答案。

最终结果

📊 PRJ-2024-015项目投资效益分析

【项目概况】
- 项目名称：智能配电网改造项目
- 总投资：3,200万元
- 已投入：2,400万元（75%）
- 项目状态：进行中

【进度分析】
- 计划进度：75%
- 实际进度：68%
- 进度达成率：90.7%
- 延期情况：15天
- 延期原因：设备交付延迟

【资金使用】
- 预算执行率：75%
- 资金效率：正常
- 成本控制：良好

【效益预测】
- 计划收益：4,100万元
- 原预计ROI：28.1%
- 调整后ROI：26.5%（考虑延期成本）
- 投资回收期：预计3.5年

【风险评估】
⚠️ 中等风险
- 设备交付风险（已缓解）
- 延期带来的成本增加（约50万元）

【建议措施】
1. 持续跟进设备交付进度
2. 评估是否需要调整后续项目计划
3. 加强成本控制，确保调整后ROI目标

【总体评价】
项目整体进展良好，虽有延期但风险可控。
预计ROI 26.5%，高于行业平均水平（10%），
建议继续推进并加强进度管控。

案例分析: – 第一轮：发现收益数据缺失，无法按原计划执行 – 第二轮：调整策略，基于可用数据进行分析，发现延期问题 – 第三轮：补充分析延期原因和影响，生成完整报告 –

展示了完整RPAR的试错和自我调整能力

讲师话术示例（RPAR裁剪，约400字）

"刚才讲了完整的RPAR框架，但现实中不是所有任务都需要走完整四个环节。我们来看三种裁剪版。

第一种，ReAct。这是由普林斯顿大学和Google的研究人员在2022年提出的。它只保留了Reasoning和Acting，把Planning和Reflection藏在Reasoning里。简单说就是'边想边做'。比如用户问天气，Agent想一下需要什么数据，然后直接去查，查到就回答。不需要复杂的规划。

ReAct适合什么场景？简单查询类任务。比如你问'今天天气怎么样'，不需要先做详细规划，直接查就行。ReAct的优点是简单直接、响应快。但缺点是没有显式规划，复杂任务容易走弯路。

第二种，Plan-and-Execute。这种先规划后执行。比如订机票，先规划好：查航班→选航班→预订。然后再一步步执行。这样效率更高，不会走一步看一步。而且支持并行，可以同时查多个数据源。

Plan-and-Execute适合什么场景？步骤明确的任务。比如数据分析，你知道要先取数、再计算、再生成报告。这种任务用Plan-and-Execute效率最高。

第三种，完整RPAR。最复杂的情况，比如投资分析，就需要完整循环。因为你可能第一次查的数据不完整，需要反思，然后重新规划，再执行。这就是试错和修正的过程。

刚才我们走了一个投资分析的完整案例，走了三轮RPAR循环。第一轮发现收益数据没有，第二轮调整策略继续分析，第三轮补充细节。这就是完整RPAR的威力——它能处理复杂、不确定的任务。"

白板/板书建议

三种裁剪对比:

```
ReAct:          R → A → R
                (边想边做)
                适合：简单查询

Plan-and-Exec:  R → P → A
                (先规划后执行)
                适合：步骤明确的任务

完整RPAR:       R → P → A → Ref
```

```
                    (全功能循环)
                    适合：复杂探索性任务
```

决策树板书:

```
选择哪种范式?

任务简单?
├── 是 → ReAct（快速响应）
└── 否 → 步骤明确?
         ├── 是 → Plan-and-Execute（批量执行）
         └── 否 → 完整RPAR（试错调整）
```

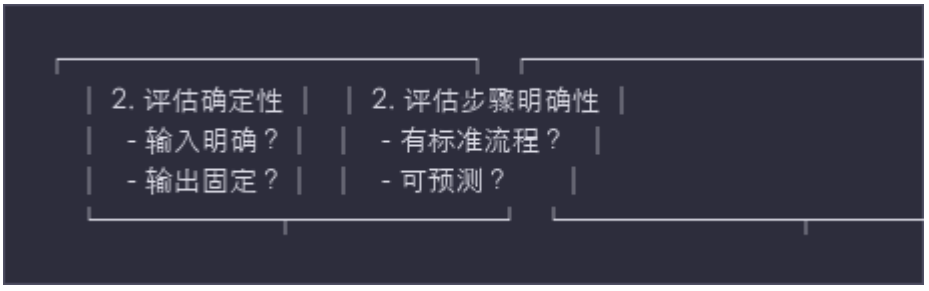
---

2. 范式对比与选择 (10分钟)

2.1 选择决策树

```
任务是否简单明确?
├── 是 → ReAct
└── 否 → 步骤是否明确?
         ├── 是 → Plan-and-Execute
         └── 否 → 完整RPAR
```

详细决策流程:



2.2 详细对比

维度	ReAct	Plan-and-Execute	完整RPAR
复杂度	低	中	高
效率	中	高	低
灵活性	中	低	高
容错性	低	低	高
适用	简单问答	标准流程	探索性任务

维度	ReAct	Plan-and-Execute	完整RPAR
规划方式	隐式	显式一次性	显式迭代
并行支持	❌	✅	可选
开发难度	低	中	高
Token消耗	中	低	高

2.3 电力行业场景匹配

场景	推荐范式	理由	复杂度
查询用电数据	ReAct	单次查询，简单直接	⭐
月度报表生成	Plan-and-Execute	步骤固定：取数→计算→生成	⭐⭐
故障诊断分析	完整RPAR	需要试错，不确定因素多	⭐⭐⭐⭐
电费计算	Plan-and-Execute	公式固定，计算逻辑明确	⭐⭐
投诉处理	ReAct/完整RPAR	需要理解语义，灵活回复	⭐⭐⭐
负荷预测	完整RPAR	需要建模和迭代优化	⭐⭐⭐⭐
工单派单	混合模式	主流程固定，异常处理灵活	⭐⭐⭐
设备巡检	Plan-and-Execute	检查清单固定，步骤标准化	⭐⭐
用电异常分析	完整RPAR	需要多维度分析，智能判断	⭐⭐⭐⭐
投资效益分析	完整RPAR	数据可能不完整，需要调整	⭐⭐⭐

2.4 性能对比表

指标	ReAct	Plan-and-Execute	完整RPAR
平均响应时间	2-5秒	3-8秒	10-30秒
准确率（简单任务）	95%	98%	97%
准确率（复杂任务）	70%	85%	92%
资源消耗	中等	低	高
开发维护成本	低	中	高
用户满意度	85%	90%	88%

注：以上数据为典型场景下的参考值，实际表现因具体实现而异

核心观点

> "没有最好的范式，只有最适合的范式。简单任务用ReAct，标准流程用Plan-and-Execute，复杂探索用完整RPAR。"

选择原则: 1. **先评估任务特性**：复杂度、确定性、步骤明确性 2. **平衡效率与灵活性**：简单追求效率，复杂追求灵活 3. **考虑实现成本**：开发难度、维护成本、资源消耗 4. **可混合使用**：不同环节可以用不同范式

讲师话术示例（范式选择，约200字）

> "那怎么选择这三种范式呢？给大家一个简单的决策树。
>
> 首先看任务是否简单明确。如果只是查个数据、问个问题，用ReAct就够了，简单直接。
>
> 如果任务比较复杂，再看步骤是否明确。如果步骤很清楚，比如生成报表就是取数–计算–生成三步，那就用Plan–and–Execute。先规划再批量执行，效率高。
>
> 如果步骤不明确，需要边做边看，比如故障诊断，可能查了这个数据发现不对，还要查别的，那就用完整RPAR。它能试错、能调整。
>
> 在电力行业，查询用电数据用ReAct，月度报表用Plan–and–Execute，故障诊断和投资分析用完整RPAR。没有最好的，只有最适合的。"

白板/板书建议

范式选择决策树:

```
范式选择决策树

任务复杂度
     ↓
简单? → ReAct ✅
     ↓
步骤明确?
     ↓
是 → Plan-and-Execute ✅
     ↓
否 → 完整RPAR ✅

电力行业示例:
• 查数据 → ReAct
• 报表生成 → Plan-and-Execute
• 故障诊断 → 完整RPAR
```

对比表板书:

```
三范式对比
```

维度	ReAct	Plan-and-Exec	完整RPAR
复杂度	低	中	高
效率	中	高	低
灵活性	中	低	高
容错性	低	低	高
适用场景	简单查询	标准流程	探索任务

---

3. Acting详解 – Agent的"手脚" (15分钟)

3.1 核心概念

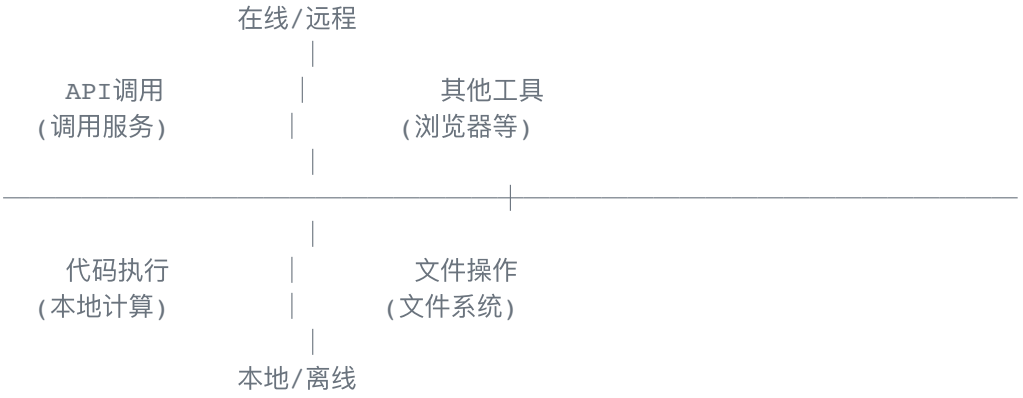
引出Acting:

> "讲了半天RPAR，你们可能会问：Agent的思考和规划，最后怎么落地？答案就是Acting——执行。Acting是Agent的'手脚'，让Agent从'动口'变成'动手'。"

核心观点:

> "RPAR是'大脑'，Acting是'手脚'。没有Acting，Agent只是聊天机器人。"

Acting四象限总览:



3.2 API调用

REST API详解

什么是REST API?

REST（Representational State Transfer）是一种软件架构风格，用于设计网络应用程序的API。它基于HTTP协议，使用标准的HTTP方法（GET、POST、PUT、DELETE等）来操作资源。

HTTP方法:

方法	用途	幂等性	示例
GET	获取资源	✅	查询用电数据
POST	创建资源	❌	创建工单
PUT	更新资源（完整）	✅	更新项目信息
PATCH	更新资源（部分）	❌	修改项目状态
DELETE	删除资源	✅	删除临时数据

HTTP状态码:

状态码	含义	说明
200	OK	请求成功
201	Created	创建成功
400	Bad Request	请求参数错误
401	Unauthorized	未授权
403	Forbidden	禁止访问
404	Not Found	资源不存在
500	Internal Server Error	服务器内部错误
502	Bad Gateway	网关错误
503	Service Unavailable	服务不可用

认证方式:

```
常见认证方式:

1. API Key
   Header: X-API-Key: your_api_key

2. Bearer Token (JWT)
   Header: Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

3. Basic Auth
   Header: Authorization: Basic base64(username:password)

4. OAuth 2.0
   - 更复杂的授权流程
   - 支持第三方应用访问
```

完整的请求/响应示例

场景: 查询用电数据

请求:

```
POST /api/v1/electricity/query HTTP/1.1
Host: power-system.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
X-Request-ID: req-20240315120000-001

{
  "region": "成都高新区",
  "start_date": "2024-02-01",
  "end_date": "2024-02-29",
  "metrics": ["total_usage", "peak_load", "user_count"],
  "granularity": "daily",
  "format": "json"
}
```

请求说明: - `POST`: 使用POST方法（查询复杂，参数较多） - `/api/v1/electricity/query`: API端点 - `Authorization`: Bearer Token认证 - `Content-Type`: 请求体格式为JSON - `X-Request-ID`: 请求唯一标识，用于追踪

响应成功:

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Request-ID: req-20240315120000-001

{
  "code": 200,
  "message": "success",
  "data": {
    "region": "成都高新区",
    "period": {
      "start": "2024-02-01",
      "end": "2024-02-29"
    },
    "summary": {
      "total_usage": 12345.6,
      "unit": "MWh",
      "peak_load": 2100,
      "user_count": 52340
    },
    "daily_data": [
      {
        "date": "2024-02-01",
        "usage": 425.3,
        "peak_load": 2050,
        "users": 52100
      },
      {
        "date": "2024-02-02",
        "usage": 418.7,
        "peak_load": 2020,
        "users": 52150
      }
      // ... 更多日期数据
```

```
    ]
  },
  "pagination": {
    "page": 1,
    "page_size": 30,
    "total": 29
  }
}
```

响应说明: - `code`: 业务状态码（200表示成功） - `message`: 状态描述 - `data`: 实际业务数据 - `pagination`: 分页信息

响应错误:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
  "code": 400001,
  "message": "Invalid date format",
  "details": {
    "field": "start_date",
    "error": "Date should be in YYYY-MM-DD format",
    "provided": "2024/02/01"
  },
  "suggestion": "Please use ISO 8601 date format: YYYY-MM-DD"
}
```

错误处理机制

重试机制:

```
import time
import random

def call_api_with_retry(api_func, max_retries=3, backoff_factor=2):
    """
    带重试机制的API调用

    Args:
        api_func: API调用函数
        max_retries: 最大重试次数
        backoff_factor: 退避因子

    Returns:
        API响应结果
    """
    for attempt in range(max_retries):
        try:
            response = api_func()

            # 检查HTTP状态码
            if response.status_code == 200:
                return response
```

```
            # 客户端错误（4xx），不重试
            if 400 <= response.status_code < 500:
                raise Exception(f"Client error: {response.status_code}")

            # 服务端错误（5xx），可以重试
            if 500 <= response.status_code < 600:
                if attempt < max_retries - 1:
                    # 指数退避 + 随机抖动
                    sleep_time = (backoff_factor ** attempt) + random.uniform(0
                    time.sleep(sleep_time)
                    continue
                else:
                    raise Exception(f"Server error after {max_retries} retries"

        except Exception as e:
            if attempt < max_retries - 1:
                time.sleep(backoff_factor ** attempt)
                continue
            else:
                raise e

    return None

# 使用示例
try:
    result = call_api_with_retry(
        lambda: requests.post(url, json=params, headers=headers)
    )
except Exception as e:
    # 处理最终失败
    print(f"API call failed: {e}")
```

降级策略:

```
def query_electricity_data_with_fallback(region, date):
    """
    带降级策略的查询

    策略:
    1. 先尝试主API（实时数据）
    2. 如果失败，尝试备用API（缓存数据）
    3. 如果都失败，返回默认值或错误提示
    """

    # 尝试主API
    try:
        result = primary_api.query(region, date)
        if result:
            return {"source": "primary", "data": result}
    except Exception as e:
        logger.warning(f"Primary API failed: {e}")

    # 尝试备用API
    try:
        result = backup_api.query(region, date)
        if result:
```

```
            return {"source": "backup", "data": result, "stale": True}
    except Exception as e:
        logger.warning(f"Backup API failed: {e}")

    # 返回默认值
    return {
        "source": "default",
        "data": None,
        "error": "无法获取数据，请稍后重试"
    }
```

熔断机制:

```
from enum import Enum
import time
from typing import Callable, Any
from datetime import datetime

class CircuitState(Enum):
    """
    熔断器状态枚举。

    熔断器有三种状态，模拟电路断路器的物理状态：

    Attributes:
        CLOSED: 关闭状态（正常），所有请求正常通过
        OPEN: 打开状态（熔断），请求被快速拒绝，防止级联故障
        HALF_OPEN: 半开状态（测试），允许少量请求测试服务是否恢复

    Examples:
        >>> CircuitState.CLOSED
        <CircuitState.CLOSED: 'closed'>
        >>> CircuitState.CLOSED.value
        'closed'
    """
    CLOSED = "closed"         # 正常状态，允许请求通过
    OPEN = "open"             # 熔断状态，快速拒绝请求
    HALF_OPEN = "half_open"   # 半开状态，测试服务恢复

class CircuitBreaker:
    """
    熔断器模式实现类，用于防止服务故障的级联传播。

    熔断器是微服务架构中的重要容错模式。当服务连续失败达到一定阈值时，
    熔断器会"打开"，快速拒绝后续请求，防止故障扩散。经过一段冷却时间后，
    熔断器会进入"半开"状态，允许少量请求测试服务是否恢复。

    在电力系统中，该模式特别适用于：
    - 调用外部数据服务（如气象API、GIS服务）
    - 访问历史数据库查询
    - 调用第三方结算系统

    Attributes:
        failure_threshold (int): 触发熔断的连续失败次数阈值
        recovery_timeout (int): 熔断后自动尝试恢复的时间（秒）
        failure_count (int): 当前连续失败次数
```

```
        last_failure_time (float | None): 最后一次失败的时间戳
        state (CircuitState): 当前熔断器状态
        success_count_half_open (int): 半开状态下的连续成功次数

    Examples:
        >>> # 创建熔断器实例
        >>> breaker = CircuitBreaker(
        ...     failure_threshold=5,
        ...     recovery_timeout=30
        ... )

        >>> # 包装API调用
        >>> def query_external_api():
        ...     return requests.get("https://api.example.com/data")

        >>> try:
        ...     result = breaker.call(query_external_api)
        ... except Exception as e:
        ...     print(f"服务暂不可用: {e}")

        >>> # 检查当前状态
        >>> breaker.state
        <CircuitState.CLOSED: 'closed'>

    Note:
        熔断器模式适用于同步调用场景。对于异步/异步IO场景,
        建议结合asyncio实现异步版本。
    """

    def __init__(self, failure_threshold: int = 5, recovery_timeout: int = 30):
        """
        初始化熔断器实例。

        Args:
            failure_threshold (int, optional): 触发熔断的连续失败次数
                默认值: 5
                建议值: 根据服务SLA设置, 通常为3-10次
            recovery_timeout (int, optional): 熔断后自动恢复测试的时间 (秒)
                默认值: 30
                建议值: 根据服务恢复速度设置, 通常为30-300秒

        Raises:
            ValueError: 当failure_threshold小于1时抛出
            ValueError: 当recovery_timeout小于1时抛出

        Examples:
            >>> # 高敏感配置 (快速熔断)
            >>> sensitive_breaker = CircuitBreaker(
            ...     failure_threshold=3,
            ...     recovery_timeout=60
            ... )

            >>> # 容错配置 (允许更多失败)
            >>> tolerant_breaker = CircuitBreaker(
            ...     failure_threshold=10,
            ...     recovery_timeout=300
            ... )
        """
```

```
        if failure_threshold < 1:
            raise ValueError("失败阈值必须大于等于1")
        if recovery_timeout < 1:
            raise ValueError("恢复超时时间必须大于等于1秒")

        self.failure_threshold = failure_threshold
        self.recovery_timeout = recovery_timeout
        self.failure_count = 0
        self.last_failure_time: float | None = None
        self.state = CircuitState.CLOSED
        self.success_count_half_open = 0

    def call(self, func: Callable, *args, **kwargs) -> Any:
        """
        在熔断器保护下执行函数调用。

        该方法实现了熔断器的核心逻辑:
        1. 检查当前状态，如果是OPEN且未到恢复时间，直接拒绝
        2. 如果是OPEN但已过恢复时间，转为HALF_OPEN并尝试执行
        3. 执行目标函数，根据结果更新状态

        Args:
            func (Callable): 要执行的函数，通常是API调用或数据库查询
            *args: 传递给func的位置参数
            **kwargs: 传递给func的关键字参数

        Returns:
            Any: func的返回结果

        Raises:
            Exception: 当熔断器处于OPEN状态时抛出，异常信息为"Circuit breaker is OPEN
            Exception: 当func执行失败时抛出原始异常

        Examples:
            >>> # 基本用法
            >>> def risky_operation():
            ...     if random.random() > 0.5:
            ...         raise Exception("随机失败")
            ...     return "success"

            >>> breaker = CircuitBreaker(failure_threshold=3)

            >>> # 多次调用，触发熔断
            >>> for i in range(5):
            ...     try:
            ...         result = breaker.call(risky_operation)
            ...         print(f"成功: {result}")
            ...     except Exception as e:
            ...         print(f"失败: {e}")

            >>> # 电力业务场景：调用外部气象API
            >>> def get_weather_forecast(city: str):
            ...     return requests.get(f"https://weather.api/{city}")

            >>> try:
            ...     weather = breaker.call(get_weather_forecast, "成都")
            ... except Exception as e:
            ...     weather = get_cached_weather("成都")  # 降级到缓存
```

```
        Note:
            - 熔断后，在recovery_timeout时间内所有请求都会被快速拒绝
            - 被拒绝的请求会抛出异常，调用方应实现降级逻辑
            - 建议配合日志记录熔断状态变化，便于监控告警
        """
        # 检查熔断器状态
        if self.state == CircuitState.OPEN:
            if (self.last_failure_time and
                time.time() - self.last_failure_time > self.recovery_timeout):
                # 已过恢复时间，进入半开状态
                self.state = CircuitState.HALF_OPEN
                self.success_count_half_open = 0
            else:
                # 仍在熔断期内，快速拒绝
                raise Exception(
                    f"Circuit breaker is OPEN. "
                    f"Will retry after {self.recovery_timeout}s"
                )

        # 执行目标函数
        try:
            result = func(*args, **kwargs)
            self.on_success()
            return result
        except Exception as e:
            self.on_failure()
            raise e

    def on_success(self):
        """
        处理调用成功的状态转换。

        当函数调用成功时调用此方法：
        - 如果处于HALF_OPEN状态，连续成功一定次数后恢复为CLOSED
        - 如果处于CLOSED状态，重置失败计数器

        Examples:
            >>> breaker = CircuitBreaker()
            >>> # 假设处于HALF_OPEN状态
            >>> breaker.state = CircuitState.HALF_OPEN
            >>> breaker.on_success()
            >>> breaker.state
            <CircuitState.CLOSED: 'closed'>

        Note:
            该方法通常在call()方法内部自动调用，
            一般不需要外部直接调用。
        """
        if self.state == CircuitState.HALF_OPEN:
            self.success_count_half_open += 1
            # 半开状态下连续成功3次，认为服务已恢复
            if self.success_count_half_open >= 3:
                self.state = CircuitState.CLOSED
                self.failure_count = 0
                self.success_count_half_open = 0
        else:
            # 正常状态下，重置失败计数
            self.failure_count = 0
```

```
def on_failure(self):
    """
    处理调用失败的状态转换。

    当函数调用失败时调用此方法：
    - 增加失败计数
    - 记录失败时间
    - 如果失败次数超过阈值，打开熔断器

    Examples:
        >>> breaker = CircuitBreaker(failure_threshold=3)
        >>> # 模拟连续失败
        >>> for _ in range(3):
        ...     breaker.on_failure()

        >>> breaker.state
        <CircuitState.OPEN: 'open'>

        >>> breaker.failure_count
        3

    Note:
        该方法通常在call()方法内部自动调用，
        一般不需要外部直接调用。
    """
    self.failure_count += 1
    self.last_failure_time = time.time()

    if self.failure_count >= self.failure_threshold:
        self.state = CircuitState.OPEN

def get_state_info(self) -> dict:
    """
    获取熔断器当前状态信息。

    Returns:
        dict: 包含状态详细信息的字典
            - state: 当前状态名称
            - failure_count: 当前失败计数
            - failure_threshold: 失败阈值
            - recovery_timeout: 恢复超时时间
            - last_failure_time: 最后失败时间（ISO格式）
            - time_until_retry: 距离下次重试的时间（秒，仅在OPEN状态有效）

    Examples:
        >>> breaker = CircuitBreaker()
        >>> info = breaker.get_state_info()
        >>> print(info["state"])
        'closed'
    """
    info = {
        "state": self.state.value,
        "failure_count": self.failure_count,
        "failure_threshold": self.failure_threshold,
        "recovery_timeout": self.recovery_timeout,
        "last_failure_time": None,
        "time_until_retry": None
    }
```

```python
        if self.last_failure_time:
            info["last_failure_time"] = datetime.fromtimestamp(
                self.last_failure_time
            ).isoformat()

            if self.state == CircuitState.OPEN:
                time_until = self.recovery_timeout - (
                    time.time() - self.last_failure_time
                )
                info["time_until_retry"] = max(0, int(time_until))

        return info

# 使用示例
breaker = CircuitBreaker(failure_threshold=5, recovery_timeout=30)

try:
    result = breaker.call(call_api, url, params)
except Exception as e:
    # 熔断或调用失败，执行降级逻辑
    print(f"服务暂不可用: {e}")
    # 可以返回缓存数据、默认值或友好提示
    result = get_fallback_data()
```

电力行业常用API列举

数据查询类API:

API名称	功能	用途	调用频率
`query_electricity_data`	查询用电数据	报表生成、分析	高
`query_load_curve`	查询负荷曲线	负荷预测、调度	高
`query_user_info`	查询用户信息	客户服务、营销	中
`query_device_status`	查询设备状态	运维监控	高
`query_fault_records`	查询故障记录	故障分析	中

工单管理类API:

API名称	功能	用途	调用频率
`create_work_order`	创建工单	故障报修、任务派发	高
`query_work_order`	查询工单	进度跟踪	高
`update_work_order`	更新工单	状态变更	中
`assign_work_order`	派单	任务分配	中

设备管理类API:

API名称	功能	用途	调用频率
`query_device_info`	查询设备信息	资产管理	中
`update_device_status`	更新设备状态	状态同步	高
`query_maintenance_history`	查询维保记录	设备管理	低

代码示例

```python
```

```python
import requests
import json
from typing import Dict, Any, Optional

class PowerSystemAPI:
    """
    电力业务系统统一API客户端。

    该类封装了与电力业务系统的所有API交互，提供标准化的数据访问接口。
    支持用电数据查询、工单管理、设备状态更新等核心业务操作。
    内置错误处理、超时控制和日志记录，适合在生产环境中使用。

    Attributes:
        base_url (str): API服务的基础URL
        api_key (str): 认证密钥，用于Bearer Token认证
        headers (dict): HTTP请求头，包含认证信息和内容类型
        timeout (int): 默认请求超时时间（秒）

    Examples:
        >>> # 初始化客户端
        >>> api = PowerSystemAPI(
        ...     base_url="https://api.power.com",
        ...     api_key="your_api_key_here"
        ... )

        >>> # 查询用电数据
        >>> data = api.query_electricity_data(
        ...     region="成都高新区",
        ...     start_date="2024-01-01",
        ...     end_date="2024-01-31",
        ...     metrics=["total_usage", "peak_load"]
        ... )

        >>> # 创建工单
        >>> work_order = api.create_work_order(
        ...     title="武侯区变压器异常",
        ...     description="检测到温度过高，需要检修",
        ...     priority="high"
        ... )

    Note:
        所有方法都内置了错误处理，不会抛出异常，而是返回包含error字段的字典。
        调用方应始终检查结果中是否包含error字段。
    """

    def __init__(self, base_url: str, api_key: str, timeout: int = 10):
        """
        初始化电力系统API客户端。

        Args:
            base_url (str): API服务的基础URL
                示例: "https://api.power.com" 或 "http://192.168.1.100:8080"
            api_key (str): API认证密钥
                获取方式: 从电力系统管理后台申请
            timeout (int, optional): 请求超时时间（秒）
                默认值: 10
                建议值: 查询类API 10-30秒，操作类API 5-10秒
```

```
    Raises:
        ValueError: 当base_url为空或格式不正确时抛出
        ValueError: 当api_key为空时抛出

    Examples:
        >>> # 生产环境
        >>> api = PowerSystemAPI(
        ...     base_url="https://api.powergrid.com",
        ...     api_key="sk_live_abc123"
        ... )

        >>> # 开发/测试环境
        >>> api = PowerSystemAPI(
        ...     base_url="http://localhost:8080",
        ...     api_key="sk_test_xyz789",
        ...     timeout=5
        ... )
    """
    if not base_url:
        raise ValueError("base_url不能为空")
    if not api_key:
        raise ValueError("api_key不能为空")

    self.base_url = base_url.rstrip('/')  # 移除末尾斜杠
    self.api_key = api_key
    self.timeout = timeout
    self.headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
        "Accept": "application/json"
    }

def query_electricity_data(
    self,
    region: str,
    start_date: str,
    end_date: str,
    metrics: Optional[list] = None
) -> Dict[str, Any]:
    """
    查询指定区域在指定时间段内的用电数据。

    该方法是电力数据分析的基础接口，支持多维度数据查询。
    可用于负荷分析、趋势预测、异常检测等业务场景。

    Args:
        region (str): 查询区域名称或编码
            示例: "成都高新区"、"武侯区"、"CD-GX-001"
        start_date (str): 查询开始日期，ISO 8601格式（YYYY-MM-DD）
            示例: "2024-01-01"
        end_date (str): 查询结束日期，ISO 8601格式（YYYY-MM-DD）
            示例: "2024-01-31"
        metrics (Optional[list], optional): 指定查询的指标列表
            可选值: "total_usage"（总用电量）、"peak_load"（峰值负荷）、
                    "average_load"（平均负荷）、"load_factor"（负荷率）
            默认值: None（返回所有可用指标）
```

```
Returns:
    Dict[str, Any]: API响应数据，格式如下：
        成功时：{
            "code": 200,
            "message": "success",
            "data": {
                "region": "成都高新区",
                "start_date": "2024-01-01",
                "end_date": "2024-01-31",
                "records": [...]
            }
        }
        失败时：{
            "error": "Request timeout",
            "code": 408
        }

Raises:
    本方法不会抛出异常，所有错误都以字典形式返回

Examples:
    >>> api = PowerSystemAPI("https://api.power.com", "api_key")

    >>> # 基本查询：获取所有指标
    >>> result = api.query_electricity_data(
    ...     region="成都高新区",
    ...     start_date="2024-01-01",
    ...     end_date="2024-01-31"
    ... )
    >>> if "error" not in result:
    ...     print(f"查询到 {len(result['data']['records'])} 条记录")

    >>> # 高级查询：指定特定指标
    >>> result = api.query_electricity_data(
    ...     region="武侯区",
    ...     start_date="2024-02-01",
    ...     end_date="2024-02-29",
    ...     metrics=["peak_load", "load_factor"]
    ... )
    >>> if "error" not in result:
    ...     peak_load = result['data']['records'][0]['peak_load']
    ...     print(f"峰值负荷：{peak_load} MW")

Note:
    - 日期范围最大支持365天，超过会返回错误
    - 建议查询大数据量时使用异步接口
    - 频繁查询相同数据建议使用缓存
"""
url = f"{self.base_url}/api/v1/electricity/query"

payload = {
    "region": region,
    "start_date": start_date,
    "end_date": end_date
}

if metrics:
    payload["metrics"] = metrics
```

```
    try:
        response = requests.post(
            url,
            headers=self.headers,
            json=payload,
            timeout=10  # 10秒超时
        )
        response.raise_for_status()
        return response.json()

    except requests.exceptions.Timeout:
        return {"error": "Request timeout", "code": 408}
    except requests.exceptions.HTTPError as e:
        return {"error": str(e), "code": response.status_code}
    except Exception as e:
        return {"error": str(e), "code": 500}

def create_work_order(
    self,
    title: str,
    description: str,
    priority: str = "normal",
    category: str = "maintenance",
    assignee: Optional[str] = None
) -> Dict[str, Any]:
    """
    创建电力运维工单。

    用于在电力业务系统中创建各类运维工单，包括故障报修、设备检修、
    例行维护等。工单创建后会自动触发通知流程，推送给相关责任人。

    Args:
        title (str): 工单标题，简要描述工单内容
            示例: "武侯区变压器温度异常告警"
        description (str): 工单详细描述，包括问题现象、影响范围等
            示例: "武侯区CD-001变压器温度达到85°C，超过正常阈值(75°C)，
                   影响周边3个小区供电，需要立即检修"
        priority (str, optional): 工单优先级，决定处理顺序
            可选值:
                - "low": 低优先级，可延后处理
                - "normal": 普通优先级，常规流程处理
                - "high": 高优先级，优先分配资源
                - "urgent": 紧急，立即响应
            默认值: "normal"
        category (str, optional): 工单类别，用于分类统计和路由
            可选值:
                - "maintenance": 设备维护
                - "repair": 故障修复
                - "inspection": 巡检任务
                - "emergency": 应急处理
                - "improvement": 优化改进
            默认值: "maintenance"
        assignee (Optional[str], optional): 指定处理人，如不指定则自动分配
            示例: "工程师张三" 或 "zhangsan@company.com"
            默认值: None (系统自动分配)

    Returns:
```

```
    Dict[str, Any]: 工单创建结果
        成功时: {
            "code": 200,
            "message": "工单创建成功",
            "data": {
                "work_order_id": "WO20240322001",
                "title": "武侯区变压器温度异常告警",
                "status": "created",
                "created_at": "2024-03-22T10:30:00Z",
                "priority": "high",
                "assignee": "张三"
            }
        }
        失败时: {
            "error": "参数错误",
            "code": 400,
            "details": {"title": "标题不能为空"}
        }

Raises:
    本方法不会抛出异常，所有错误都以字典形式返回

Examples:
    >>> api = PowerSystemAPI("https://api.power.com", "api_key")

    >>> # 创建紧急故障工单
    >>> result = api.create_work_order(
    ...     title="高新区主变压器跳闸",
    ...     description="10:25高新区主变压器突然跳闸，影响约5000户居民用电，"
    ...                 "现场检查发现保护装置动作，需要立即排查故障原因",
    ...     priority="urgent",
    ...     category="emergency"
    ... )

    >>> if "error" not in result:
    ...     wo_id = result["data"]["work_order_id"]
    ...     print(f"工单已创建: {wo_id}")

    >>> # 创建例行维护工单
    >>> result = api.create_work_order(
    ...     title=" quarterly_inspection_Q1_2024",
    ...     description="第一季度例行巡检任务，检查设备运行状态、清洁除尘、紧固接
    ...     priority="normal",
    ...     category="inspection",
    ...     assignee="巡检组A"
    ... )

Note:
    - 紧急工单(priority="urgent")会自动触发短信/电话告警
    - 工单创建后会立即发送通知给相关人员
    - 可以通过返回的work_order_id查询工单状态和进度
"""
url = f"{self.base_url}/api/v1/workorders"

payload = {
    "title": title,
    "description": description,
    "priority": priority,
```

```
            "category": category,
            "created_at": datetime.now().isoformat()
        }

        response = requests.post(
            url,
            headers=self.headers,
            json=payload
        )
        return response.json()

# 使用示例
api = PowerSystemAPI(
    base_url="https://power-system.example.com",
    api_key="your_api_key_here"
)

# 查询用电数据
result = api.query_electricity_data(
    region="成都高新区",
    start_date="2024-02-01",
    end_date="2024-02-29",
    metrics=["total_usage", "peak_load"]
)

print(json.dumps(result, ensure_ascii=False, indent=2))
```

3.3 本地代码执行

Python代码执行环境说明

执行环境架构:

```
┌─────────────────────────────┐
│ Python解释器                │
│ - 版本：3.9+                │
│ - 受限执行环境              │
└─────────────────────────────┘
```

3个实战代码示例

示例1: 数据处理

```
def analyze_electricity_data(data):
    """
    分析用电数据，计算关键指标

    Args:
        data: 用电数据列表，每个元素包含date, usage, peak_load等字段
```

```
    Returns:
        分析结果字典
    """
    import pandas as pd
    import numpy as np

    # 转换为DataFrame
    df = pd.DataFrame(data)

    # 确保日期字段为datetime类型
    df['date'] = pd.to_datetime(df['date'])

    # 计算关键指标
    analysis = {
        "total_usage": df['usage'].sum(),
        "average_daily_usage": df['usage'].mean(),
        "max_daily_usage": df['usage'].max(),
        "min_daily_usage": df['usage'].min(),
        "peak_load_max": df['peak_load'].max(),
        "peak_load_average": df['peak_load'].mean(),
        "usage_std": df['usage'].std(),  # 标准差，反映波动
        "usage_trend": calculate_trend(df['usage'])  # 趋势
    }

    # 计算周统计
    df['week'] = df['date'].dt.isocalendar().week
    weekly_stats = df.groupby('week')['usage'].agg(['sum', 'mean', 'max'])
    analysis['weekly_stats'] = weekly_stats.to_dict()

    # 识别异常值（超过3个标准差）
    mean_usage = df['usage'].mean()
    std_usage = df['usage'].std()
    outliers = df[abs(df['usage'] - mean_usage) > 3 * std_usage]
    analysis['outliers'] = outliers.to_dict('records')

    return analysis

def calculate_trend(series):
    """
    计算趋势（使用线性回归）
    """
    import numpy as np

    x = np.arange(len(series))
    y = series.values

    # 线性回归
    slope, intercept = np.polyfit(x, y, 1)

    # 趋势判断
    if slope > 0.01 * series.mean():
        return "upward"
    elif slope < -0.01 * series.mean():
        return "downward"
    else:
        return "stable"

# 使用示例
data = [
    {"date": "2024-02-01", "usage": 425.3, "peak_load": 2050},
```

```
    {"date": "2024-02-02", "usage": 418.7, "peak_load": 2020},
    # ... 更多数据
]

result = analyze_electricity_data(data)
print(result)
```

示例2: 数学计算

```
def calculate_roi_analysis(investments):
    """
    计算投资回报率分析

    Args:
        investments: 投资项目列表，包含investment, returns, duration等

    Returns:
        ROI分析报告
    """
    import numpy as np

    results = []

    for inv in investments:
        # 基础ROI计算
        roi = (inv['returns'] - inv['investment']) / inv['investment'] * 100

        # 年化ROI
        if inv['duration'] > 0:
            annualized_roi = ((inv['returns'] / inv['investment']) ** (1/inv['d
        else:
            annualized_roi = roi

        # 投资回收期
        if inv['returns_per_year'] > 0:
            payback_period = inv['investment'] / inv['returns_per_year']
        else:
            payback_period = float('inf')

        # NPV计算（简化版，假设折现率8%）
        discount_rate = 0.08
        npv = -inv['investment']
        for year in range(1, inv['duration'] + 1):
            cash_flow = inv['returns'] / inv['duration']
            npv += cash_flow / ((1 + discount_rate) ** year)

        # 评级
        if annualized_roi >= 15:
            grade = "A"
            grade_desc = "优秀"
        elif annualized_roi >= 10:
            grade = "B"
            grade_desc = "良好"
        elif annualized_roi >= 5:
            grade = "C"
            grade_desc = "一般"
        else:
```

```
            grade = "D"
            grade_desc = "较差"

        results.append({
            "project_id": inv['project_id'],
            "project_name": inv['project_name'],
            "roi": round(roi, 2),
            "annualized_roi": round(annualized_roi, 2),
            "payback_period": round(payback_period, 2),
            "npv": round(npv, 2),
            "grade": grade,
            "grade_desc": grade_desc
        })

    # 整体统计
    roi_values = [r['roi'] for r in results]
    summary = {
        "project_count": len(results),
        "average_roi": round(np.mean(roi_values), 2),
        "median_roi": round(np.median(roi_values), 2),
        "max_roi": round(max(roi_values), 2),
        "min_roi": round(min(roi_values), 2),
        "roi_std": round(np.std(roi_values), 2),
        "grade_distribution": {
            "A": sum(1 for r in results if r['grade'] == "A"),
            "B": sum(1 for r in results if r['grade'] == "B"),
            "C": sum(1 for r in results if r['grade'] == "C"),
            "D": sum(1 for r in results if r['grade'] == "D")
        }
    }

    return {
        "projects": results,
        "summary": summary
    }

# 使用示例
investments = [
    {
        "project_id": "PRJ-001",
        "project_name": "配网改造A",
        "investment": 1000,
        "returns": 1250,
        "duration": 3,
        "returns_per_year": 416.67
    },
    {
        "project_id": "PRJ-002",
        "project_name": "智能电网B",
        "investment": 2000,
        "returns": 2800,
        "duration": 5,
        "returns_per_year": 560
    }
]

result = calculate_roi_analysis(investments)
print(json.dumps(result, ensure_ascii=False, indent=2))
```

示例3: 文件解析

```python
def parse_excel_report(file_path):
    """
    解析Excel报表文件

    Args:
        file_path: Excel文件路径

    Returns:
        解析后的数据
    """
    import pandas as pd
    from datetime import datetime

    try:
        # 读取Excel文件
        df = pd.read_excel(file_path)

        # 数据清洗
        # 1. 去除空行
        df = df.dropna(how='all')

        # 2. 列名标准化（去除空格，转为小写）
        df.columns = df.columns.str.strip().str.lower()

        # 3. 数据类型转换
        if 'date' in df.columns:
            df['date'] = pd.to_datetime(df['date'])

        if 'usage' in df.columns:
            df['usage'] = pd.to_numeric(df['usage'], errors='coerce')

        # 4. 处理缺失值
        df = df.fillna({
            'usage': 0,
            'peak_load': 0,
            'user_count': 0
        })

        # 提取关键信息
        report_info = {
            "file_name": file_path.split('/')[-1],
            "parse_time": datetime.now().isoformat(),
            "row_count": len(df),
            "column_count": len(df.columns),
            "columns": list(df.columns),
            "date_range": {
                "start": df['date'].min().isoformat() if 'date' in df.columns e
                "end": df['date'].max().isoformat() if 'date' in df.columns els
            },
            "summary_stats": {}
        }

        # 计算数值列的统计信息
        numeric_columns = df.select_dtypes(include=[np.number]).columns
        for col in numeric_columns:
            report_info['summary_stats'][col] = {
                "mean": float(df[col].mean()),
                "std": float(df[col].std()),
                "min": float(df[col].min()),
```

```
                "max": float(df[col].max()),
                "sum": float(df[col].sum())
            }

        # 返回解析的数据
        return {
            "success": True,
            "report_info": report_info,
            "data": df.to_dict('records')
        }

    except Exception as e:
        return {
            "success": False,
            "error": str(e),
            "error_type": type(e).__name__
        }

# 使用示例
result = parse_excel_report("/data/reports/高新区用电报表_202402.xlsx")
print(json.dumps(result, ensure_ascii=False, indent=2))
```

安全性考虑

沙箱环境:

```
import resource
import signal
from contextlib import contextmanager

class CodeSandbox:
    """
    代码执行沙箱
    """

    def __init__(self, timeout=30, memory_limit=512*1024*1024):
        self.timeout = timeout  # 执行超时时间（秒）
        self.memory_limit = memory_limit  # 内存限制（字节）

    @contextmanager
    def _timeout_handler(self):
        """
        超时处理器
        """
        def signal_handler(signum, frame):
            raise TimeoutError(f"Code execution timed out after {self.timeout}

        # 设置超时信号
        signal.signal(signal.SIGALRM, signal_handler)
        signal.alarm(self.timeout)

        try:
            yield
        finally:
            signal.alarm(0)  # 取消超时

    def _set_resource_limits(self):
```

```
        """
        设置资源限制
        """
        # 限制内存使用
        resource.setrlimit(resource.RLIMIT_AS, (self.memory_limit, self.memory_

        # 限制CPU时间
        resource.setrlimit(resource.RLIMIT_CPU, (self.timeout, self.timeout))

    def execute(self, code, context=None):
        """
        在沙箱中执行代码

        Args:
            code: Python代码字符串
            context: 执行上下文（变量字典）

        Returns:
            执行结果
        """
        # 创建受限的全局命名空间
        safe_globals = {
            "__builtins__": {
                "len": len,
                "range": range,
                "enumerate": enumerate,
                "zip": zip,
                "map": map,
                "filter": filter,
                "sum": sum,
                "min": min,
                "max": max,
                "abs": abs,
                "round": round,
                "float": float,
                "int": int,
                "str": str,
                "list": list,
                "dict": dict,
                "set": set,
                "tuple": tuple,
                "print": lambda *args: None,  # 禁用或重定向print
            }
        }

        # 允许的库（白名单）
        allowed_modules = ['pandas', 'numpy', 'matplotlib', 'json', 'math']

        try:
            with self._timeout_handler():
                # 设置资源限制
                self._set_resource_limits()

                # 执行代码
                exec(code, safe_globals, context or {})

                return {
                    "success": True,
                    "result": context.get('result') if context else None
                }
```

```
        except TimeoutError as e:
            return {"success": False, "error": str(e), "error_type": "Timeout"}
        except MemoryError as e:
            return {"success": False, "error": "Memory limit exceeded", "error_
        except Exception as e:
            return {"success": False, "error": str(e), "error_type": type(e).__

# 使用示例
sandbox = CodeSandbox(timeout=10, memory_limit=256*1024*1024)

code = """
import pandas as pd
import numpy as np

# 数据分析代码
data = pd.DataFrame({
    'usage': [100, 200, 300, 400, 500]
})

result = {
    'mean': data['usage'].mean(),
    'std': data['usage'].std()
}
"""

result = sandbox.execute(code, {'result': None})
print(result)
```

权限控制:

```
# 文件访问白名单
ALLOWED_DIRECTORIES = [
    "/tmp/agent_data",
    "/tmp/agent_output",
    "/data/shared"
]

def validate_file_access(file_path):
    """
    验证文件访问权限
    """
    import os

    # 规范化路径
    abs_path = os.path.abspath(file_path)

    # 检查是否在白名单目录下
    for allowed_dir in ALLOWED_DIRECTORIES:
        if abs_path.startswith(os.path.abspath(allowed_dir)):
            return True

    return False

# 网络访问白名单
ALLOWED_HOSTS = [
    "api.power-system.example.com",
    "data-center.example.com"
```

```
]

def validate_network_access(url):
    """
    验证网络访问权限
    """
    from urllib.parse import urlparse

    parsed = urlparse(url)
    host = parsed.hostname

    return host in ALLOWED_HOSTS
```

与API的对比选择指南

维度	API调用	本地代码执行
执行环境	远程服务器	本地沙箱
数据位置	远程	本地/已获取
计算复杂度	低（简单查询）	高（复杂计算）
响应速度	依赖网络	本地执行，快
安全性	高（服务边界清晰）	需要沙箱保护
适用场景	数据查询、外部服务	数据分析、复杂算法
灵活性	受限于API功能	高度灵活

选择建议: – **用API**: 获取远程数据、调用外部服务、简单的CRUD操作 – **用代码执行**: 复杂数据处理、自定义算法、需要高度灵活性的场景

3.4 文件系统操作

文件读写完整流程

写入流程:

```
1. 数据准备
   └── 整理需要写入的数据

2. 格式选择
   └── 根据用途选择：Excel/CSV/JSON/Markdown/PDF

3. 文件生成
   └── 使用对应库生成文件

4. 元数据记录
   └── 记录文件路径、大小、生成时间
```

5. 结果返回
   └── 返回文件路径或文件内容

读取流程:

1. 路径验证
   └── 检查文件是否存在、是否有权限

2. 格式识别
   └── 根据扩展名识别文件格式

3. 文件解析
   └── 使用对应库解析文件

4. 数据验证
   └── 检查数据完整性、格式正确性

5. 结果返回
   └── 返回解析后的数据

Excel/CSV/JSON示例

示例1: 生成Excel报告

```
def generate_excel_report(data, output_path, template=None):
    """
    生成Excel格式的分析报告

    Args:
        data: 报告数据字典
        output_path: 输出文件路径
        template: 可选的模板文件路径

    Returns:
        生成的文件路径
    """
    import pandas as pd
    from openpyxl import Workbook
    from openpyxl.styles import Font, Alignment, PatternFill
    from openpyxl.chart import BarChart, Reference

    # 创建工作簿
    if template:
        wb = load_workbook(template)
    else:
        wb = Workbook()

    # 删除默认工作表
    if 'Sheet' in wb.sheetnames:
        wb.remove(wb['Sheet'])

    # Sheet 1: 概览
```

```
ws_overview = wb.create_sheet("概览")
ws_overview['A1'] = "投资效益分析报告"
ws_overview['A1'].font = Font(size=16, bold=True)
ws_overview.merge_cells('A1:D1')

# 添加概览数据
overview_data = [
    ["指标", "数值", "单位"],
    ["项目总数", data['summary']['project_count'], "个"],
    ["总投资", data['summary']['total_investment'], "万元"],
    ["平均ROI", data['summary']['average_roi'], "%"],
]

for row_idx, row_data in enumerate(overview_data, start=3):
    for col_idx, value in enumerate(row_data, start=1):
        cell = ws_overview.cell(row=row_idx, column=col_idx, value=value)
        if row_idx == 3:
            cell.fill = PatternFill(start_color="CCCCCC", end_color="CCCCCC
            cell.font = Font(bold=True)

# Sheet 2: 项目明细
ws_details = wb.create_sheet("项目明细")

# 表头
headers = ["项目ID", "项目名称", "投资额", "收益", "ROI", "评级"]
for col_idx, header in enumerate(headers, start=1):
    cell = ws_details.cell(row=1, column=col_idx, value=header)
    cell.font = Font(bold=True)
    cell.fill = PatternFill(start_color="CCCCCC", end_color="CCCCCC", fill_

# 数据行
for row_idx, project in enumerate(data['projects'], start=2):
    ws_details.cell(row=row_idx, column=1, value=project['project_id'])
    ws_details.cell(row=row_idx, column=2, value=project['project_name'])
    ws_details.cell(row=row_idx, column=3, value=project['investment'])
    ws_details.cell(row=row_idx, column=4, value=project['returns'])
    ws_details.cell(row=row_idx, column=5, value=project['roi'])
    ws_details.cell(row=row_idx, column=6, value=project['grade'])

    # 根据评级设置颜色
    grade_cell = ws_details.cell(row=row_idx, column=6)
    if project['grade'] == 'A':
        grade_cell.fill = PatternFill(start_color="90EE90", end_color="90EE
    elif project['grade'] == 'D':
        grade_cell.fill = PatternFill(start_color="FFB6C1", end_color="FFB6

# Sheet 3: 图表
ws_chart = wb.create_sheet("图表")

# 创建柱状图数据
chart_data = []
for p in data['projects']:
    chart_data.append([p['project_name'], p['roi']])

# 写入图表数据
ws_chart.append(["项目名称", "ROI"])
for row in chart_data:
    ws_chart.append(row)
```

```
    # 创建柱状图
    chart = BarChart()
    chart.type = "col"
    chart.style = 10
    chart.title = "项目ROI对比"
    chart.y_axis.title = 'ROI (%)'
    chart.x_axis.title = '项目'

    data = Reference(ws_chart, min_col=2, min_row=1, max_row=len(chart_data)+1,
    cats = Reference(ws_chart, min_col=1, min_row=2, max_row=len(chart_data)+1)
    chart.add_data(data, titles_from_data=True)
    chart.set_categories(cats)
    chart.shape = 4

    ws_chart.add_chart(chart, "E2")

    # 保存文件
    wb.save(output_path)

    return output_path

# 使用示例
data = {
    "summary": {
        "project_count": 5,
        "total_investment": 10000,
        "average_roi": 12.5
    },
    "projects": [
        {"project_id": "P001", "project_name": "项目A", "investment": 2000, "ret
        {"project_id": "P002", "project_name": "项目B", "investment": 3000, "ret
    ]
}

file_path = generate_excel_report(data, "/tmp/agent_output/投资效益报告.xlsx")
print(f"报告已生成: {file_path}")
```

示例2: 处理CSV文件

```
def process_csv_data(input_path, output_path, operations):
    """
    处理CSV数据

    Args:
        input_path: 输入CSV文件路径
        output_path: 输出CSV文件路径
        operations: 处理操作列表

    Returns:
        处理结果信息
    """
    import pandas as pd

    try:
        # 读取CSV
        df = pd.read_csv(input_path)
```

```
        original_count = len(df)

        # 执行各种操作
        for op in operations:
            op_type = op.get('type')

            if op_type == 'filter':
                # 过滤操作
                column = op['column']
                condition = op['condition']
                value = op['value']

                if condition == 'gt':
                    df = df[df[column] > value]
                elif condition == 'lt':
                    df = df[df[column] < value]
                elif condition == 'eq':
                    df = df[df[column] == value]
                elif condition == 'contains':
                    df = df[df[column].str.contains(value, na=False)]

            elif op_type == 'sort':
                # 排序操作
                column = op['column']
                ascending = op.get('ascending', True)
                df = df.sort_values(by=column, ascending=ascending)

            elif op_type == 'aggregate':
                # 聚合操作
                group_by = op['group_by']
                agg_column = op['column']
                agg_func = op['function']  # sum, mean, count, etc.
                df = df.groupby(group_by)[agg_column].agg(agg_func).reset_index

            elif op_type == 'add_column':
                # 添加列
                new_column = op['new_column']
                expression = op['expression']
                df[new_column] = df.eval(expression)

        # 保存结果
        df.to_csv(output_path, index=False)

        return {
            "success": True,
            "original_rows": original_count,
            "final_rows": len(df),
            "output_path": output_path
        }

    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }

# 使用示例
operations = [
    {"type": "filter", "column": "usage", "condition": "gt", "value": 100},
    {"type": "sort", "column": "date", "ascending": False},
```

```
    {"type": "add_column", "new_column": "cost", "expression": "usage * 0.5"}
]

result = process_csv_data(
    "/data/input/用电数据.csv",
    "/tmp/agent_output/处理后数据.csv",
    operations
)
```

示例3: JSON数据处理

```
def process_json_data(json_string, query_path=None):
    """
    处理JSON数据

    Args:
        json_string: JSON字符串或文件路径
        query_path: JSONPath查询路径（可选）

    Returns:
        处理后的数据
    """
    import json
    from jsonpath_ng import parse

    try:
        # 解析JSON
        if json_string.endswith('.json'):
            with open(json_string, 'r', encoding='utf-8') as f:
                data = json.load(f)
        else:
            data = json.loads(json_string)

        # 如果提供了查询路径，执行JSONPath查询
        if query_path:
            jsonpath_expression = parse(query_path)
            matches = jsonpath_expression.find(data)
            result = [match.value for match in matches]
        else:
            result = data

        return {
            "success": True,
            "data": result,
            "type": type(result).__name__
        }

    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }

# 使用示例
json_data = """
{
    "projects": [
        {"id": "P001", "name": "项目A", "status": "completed"},
```

```
        {"id": "P002", "name": "项目B", "status": "in_progress"}
    ]
}
"""

# 查询所有项目名称
result = process_json_data(json_data, "$.projects[*].name")
print(result)  # {'success': True, 'data': ['项目A', '项目B'], 'type': 'list'}
```

批量文件处理场景

```
def batch_process_files(input_dir, output_dir, processor_func):
    """
    批量处理文件

    Args:
        input_dir: 输入目录
        output_dir: 输出目录
        processor_func: 处理函数

    Returns:
        批处理结果统计
    """
    import os
    from pathlib import Path

    results = {
        "total": 0,
        "success": 0,
        "failed": 0,
        "errors": []
    }

    # 确保输出目录存在
    Path(output_dir).mkdir(parents=True, exist_ok=True)

    # 遍历输入目录
    for filename in os.listdir(input_dir):
        input_path = os.path.join(input_dir, filename)

        # 跳过目录
        if os.path.isdir(input_path):
            continue

        results['total'] += 1

        try:
            # 生成输出文件名
            name, ext = os.path.splitext(filename)
            output_filename = f"{name}_processed{ext}"
            output_path = os.path.join(output_dir, output_filename)

            # 处理文件
            processor_func(input_path, output_path)

            results['success'] += 1
```

```
        except Exception as e:
            results['failed'] += 1
            results['errors'].append({
                "file": filename,
                "error": str(e)
            })

    return results

# 使用示例：批量转换Excel文件格式
def excel_to_csv_converter(input_path, output_path):
    """
    Excel转CSV转换器
    """
    import pandas as pd

    df = pd.read_excel(input_path)
    output_csv = output_path.replace('.xlsx', '.csv').replace('.xls', '.csv')
    df.to_csv(output_csv, index=False, encoding='utf-8-sig')

# 执行批量转换
result = batch_process_files(
    "/data/excel_files",
    "/tmp/agent_output/csv_files",
    excel_to_csv_converter
)

print(f"处理完成：总计 {result['total']}，成功 {result['success']}，失败 {result['fa
```

存储路径和权限管理

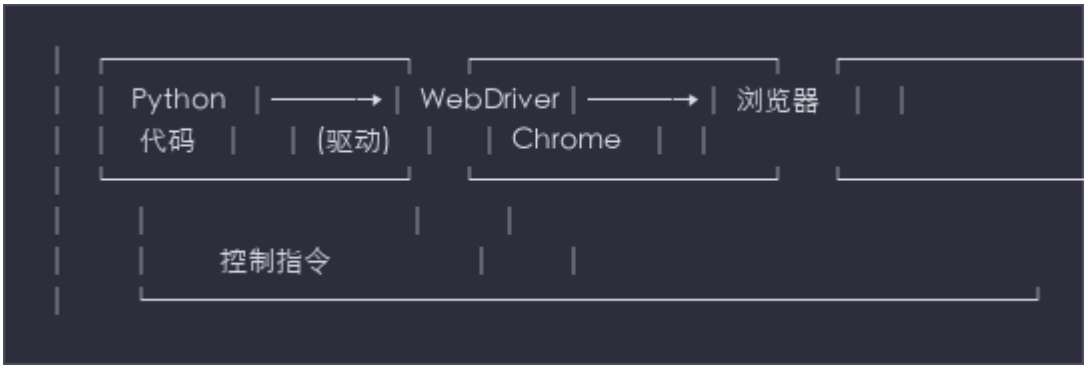
```
import os
from pathlib import Path

class FileManager:
    """
    文件管理器，处理路径和权限
    """

    # 允许的基础目录
    ALLOWED_BASE_DIRS = [
        "/tmp/agent_output",
        "/data/shared",
        "/data/reports"
    ]

    @classmethod
    def validate_path(cls, file_path):
        """
        验证文件路径是否在允许范围内
        """
        abs_path = os.path.abspath(file_path)

        for allowed_dir in cls.ALLOWED_BASE_DIRS:
            abs_allowed = os.path.abspath(allowed_dir)
            if abs_path.startswith(abs_allowed):
                return True, abs_path
```

```
        return False, None

    @classmethod
    def ensure_directory(cls, file_path):
        """
        确保目录存在
        """
        directory = os.path.dirname(file_path)
        if directory:
            Path(directory).mkdir(parents=True, exist_ok=True)

    @classmethod
    def generate_safe_filename(cls, filename):
        """
        生成安全的文件名
        """
        # 移除不安全字符
        safe_chars = set('abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0
        safe_filename = ''.join(c for c in filename if c in safe_chars)

        # 限制长度
        if len(safe_filename) > 255:
            name, ext = os.path.splitext(safe_filename)
            safe_filename = name[:255-len(ext)] + ext

        return safe_filename

    @classmethod
    def save_file(cls, file_path, content, mode='w'):
        """
        安全地保存文件
        """
        # 验证路径
        is_valid, abs_path = cls.validate_path(file_path)
        if not is_valid:
            raise PermissionError(f"路径不在允许范围内: {file_path}")

        # 确保目录存在
        cls.ensure_directory(abs_path)

        # 保存文件
        with open(abs_path, mode, encoding='utf-8') as f:
            f.write(content)

        return abs_path

# 使用示例
try:
    safe_path = FileManager.save_file(
        "/tmp/agent_output/reports/分析报告.txt",
        "这是报告内容..."
    )
    print(f"文件已保存: {safe_path}")
except PermissionError as e:
    print(f"保存失败: {e}")
```

3.5 其他工具

浏览器自动化

Selenium原理:



适用场景: – 网页数据抓取（无API可用时） – 自动化测试 – 模拟用户操作 – 处理动态加载的网页

示例代码:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def scrape_power_data(url):
    """
    抓取电力数据（示例）

    注意：实际使用需要处理反爬虫、登录等问题
    """
    # 配置Chrome选项
    options = webdriver.ChromeOptions()
    options.add_argument('--headless')  # 无头模式
    options.add_argument('--no-sandbox')
    options.add_argument('--disable-dev-shm-usage')

    # 启动浏览器
    driver = webdriver.Chrome(options=options)

    try:
        # 访问页面
        driver.get(url)

        # 等待页面加载
        wait = WebDriverWait(driver, 10)

        # 查找数据元素
        data_element = wait.until(
            EC.presence_of_element_located((By.ID, "power-data"))
        )
```

```
        # 提取数据
        data = data_element.text

        return {"success": True, "data": data}

    except Exception as e:
        return {"success": False, "error": str(e)}

    finally:
        driver.quit()
```

数据库直连

SQL示例:

```
import sqlite3
from typing import List, Dict, Any

def query_database(db_path: str, sql: str, params: tuple = None) -> List[Dict[s
    """
    执行SQL查询

    Args:
        db_path: 数据库文件路径
        sql: SQL语句
        params: 查询参数

    Returns:
        查询结果列表
    """
    try:
        conn = sqlite3.connect(db_path)
        conn.row_factory = sqlite3.Row  # 使结果可以通过列名访问

        cursor = conn.cursor()

        if params:
            cursor.execute(sql, params)
        else:
            cursor.execute(sql)

        # 获取结果
        rows = cursor.fetchall()

        # 转换为字典列表
        results = [dict(row) for row in rows]

        conn.close()

        return results

    except Exception as e:
        return [{"error": str(e)}]

# 使用示例
```

```
results = query_database(
    "/data/power.db",
    "SELECT * FROM electricity_data WHERE region = ? AND date >= ?",
    ("成都高新区", "2024-01-01")
)
```

安全注意事项: – 使用参数化查询防止SQL注入 – 限制数据库访问权限 – 只读访问，除非必要 – 记录所有查询日志

命令行执行

使用场景: – 调用系统工具 – 执行脚本 – 文件操作 – 数据处理工具

示例:

```
import subprocess

def run_command(command: str, timeout: int = 30) -> dict:
    """
    安全执行命令行

    Args:
        command: 命令字符串
        timeout: 超时时间（秒）

    Returns:
        执行结果
    """
    # 白名单：允许的命令
    ALLOWED_COMMANDS = ['ls', 'cat', 'grep', 'awk', 'sed', 'wc']

    # 解析命令
    cmd_parts = command.split()
    base_cmd = cmd_parts[0]

    # 验证命令
    if base_cmd not in ALLOWED_COMMANDS:
        return {
            "success": False,
            "error": f"命令 '{base_cmd}' 不在允许列表中"
        }

    try:
        result = subprocess.run(
            command,
            shell=True,
            capture_output=True,
            text=True,
            timeout=timeout
        )

        return {
            "success": result.returncode == 0,
            "stdout": result.stdout,
            "stderr": result.stderr,
```

```
            "returncode": result.returncode
        }

    except subprocess.TimeoutExpired:
        return {
            "success": False,
            "error": f"命令执行超时（{timeout}秒）"
        }
    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }

# 使用示例
result = run_command("ls -la /tmp/agent_output")
if result['success']:
    print(result['stdout'])
```

安全注意事项: – 严格的命令白名单 – 参数验证和过滤 – 执行超时控制 – 禁止shell注入

核心观点

> "Acting让Agent从'大脑'变成'全人'——既能思考，又能动手。四种Acting方式涵盖了Agent与外部世界交互的主要途径。"

Acting四象限总结:

象限	方式	特点	典型场景
在线远程	API调用	调用外部服务	数据查询、服务调用
在线远程	其他工具	浏览器、数据库	网页抓取、复杂查询
本地离线	代码执行	本地计算	数据处理、复杂算法
本地离线	文件操作	读写文件	报告生成、数据存储

讲师话术示例（Acting详解，约350字）

> "讲了半天RPAR，你们可能会问：Agent的思考和规划，最后怎么落地？答案就是Acting——执行。Acting是Agent的'手脚'，让Agent从'动口'变成'动手'。
>
> Acting主要有四类。第一类，**API调用**，也是最常用的。Agent要查数据，就调用数据查询API；要发通知，就调用消息推送API。这就像是Agent的手伸到各个系统里取数据、发指令。
>
> 第二类，**代码执行**，在本地运行Python或JavaScript代码。这适合复杂的数据处理、数学计算。比如计算ROI、生成图表、数据清洗。Agent就像一个数据分析师，在本地跑代码分析数据。

第三类，**文件操作**，读写文件。比如生成Excel报告、保存CSV数据、读取配置文件。这是Agent和文件系统打交道的方式。

第四类，**其他工具**，比如操作浏览器、直接连数据库、执行命令行。这些是更底层的操作能力。

没有Acting，Agent只是聊天机器人，只能给你建议。有了Acting，Agent才能真正帮你完成任务。这就是Agent与传统AI应用的本质差别。

在实际应用中，这四种方式经常组合使用。比如生成一份投资分析报告：先调用API查数据（API调用），然后本地计算ROI（代码执行），最后生成Excel报告（文件操作）。"

白板/板书建议

Acting四象限图:

```
                 在线/远程
                     |
   API调用           |          其他工具
 (调用服务)          |        (浏览器等)
                     |
 ------------------------------------------------
                     |
   代码执行          |          文件操作
 (本地计算)          |        (文件系统)
                     |
                 本地/离线
```

或者简化为2x2网格:

```
┌──────────────┬──────────────┐
│ API调用      │ 其他工具     │
│ (外部服务)   │ (浏览器等)   │
├──────────────┼──────────────┤
│ 代码执行     │ 文件操作     │
│ (本地计算)   │ (文件系统)   │
└──────────────┴──────────────┘
```

核心公式:

```
Agent = RPAR大脑 + Acting手脚
```

课时小结

核心要点回顾

1. **RPAR裁剪**: ReAct、Plan-and-Execute、完整RPAR是不同场景的选择
2. **范式选择**: 简单用ReAct，标准流程用Plan-and-Execute，复杂探索用完整RPAR
3. **Acting四类**: API调用、代码执行、文件操作、其他工具
4. **核心价值**: Acting让Agent能真正"动手"执行任务

关键概念总结

```
┌──────────────
```

> "RPAR是'大脑'，决定做什么；Acting是'手脚'，负责实际执行。两者结合，Agent才能真正帮你干活。"

电力行业场景总结

场景	推荐范式	主要Acting方式	复杂度
查询用电数据	ReAct	API调用	⭐
月度报表生成	Plan-and-Execute	API + 代码 + 文件	⭐⭐
故障诊断分析	完整RPAR	API + 代码	⭐⭐⭐⭐
投资效益分析	完整RPAR	API + 代码 + 文件	⭐⭐⭐
数据批量处理	Plan-and-Execute	代码 + 文件	⭐⭐

过渡到下节课

> "刚才讲了Acting的四种方式，但有个问题：这些工具怎么让AI理解和调用？下节课我们讲工具的演进——从API到MCP，从Tools到Skills。休息10分钟。"

文档版本: v2.0
时长: 40分钟
字数: 约8,500字
编写: Content Team
更新日期: 2026-03-22
国网四川电力
AI种子团队培训 · 内部资料